KV-Cache Quantization on NVIDIA Jetson Orin Nano

Energy, speed, memory and accuracy of f16, 8-bit and 4-bit KV caches: a complete record of what was done, how, and what was found

Koushik Bhupathiraju · Supervisors: Prof. Taskin Kocak, Prof. Rongyu Lin ·
Experiments run 19 October 2026

Contents

1. Summary

2. Background: the thesis and why the edge

3. What happened, step by step

4. Hardware

5. Software, libraries and tools

6. Models

7. KV cache formats compared

8. How energy was measured

9. Benchmark protocol and parameters

10. Accuracy test

11. Results

12. Data quality

13. Limitations

14. Next steps

Appendix A. Reproduction commands

Appendix B. Files produced

Appendix C. Problems hit along the way and how they were fixed

Appendix D. The thesis-data audit (27 September)

Appendix E. Glossary

1. Summary

This report documents the first edge-device experiment for the paper planned from the thesis *Energy Benchmarking of KV Cache Compression Methods on GPU*. On a Jetson Orin Nano Developer Kit, it measured how much energy it takes to generate text when the KV cache is stored in three formats: f16 (no compression), q8_0 (8-bit) and q4_0 (4-bit). It used two models (Llama-3.2-1B-Instruct and Qwen2.5-0.5B-Instruct), four prompt lengths (1K, 4K, 8K and 16K tokens) and two power modes (15W and 7W), with three shuffled repeats of every setting: **144 energy measurements** in total, plus six accuracy (perplexity) tests.

Headline: on this board and software, quantizing the KV cache **trades energy for memory**. It shrinks the cache 1.9 to 3.6 times, but every generated token costs more energy, and the extra cost grows with context length and with lower GPU clocks.

#	Finding	Key numbers
1	Quantized KV caches cost more decode energy per token, increasingly so at long context.	At 16K tokens: +29% to +59% at 15W, +52% to +109% at 7W. At 1K: at most +8%.
2	The memory saving is real.	KV cache at 16K tokens: Llama 516 → 274 → 145 MiB; Qwen 194 → 103 → 54 MiB.
3	8-bit is accuracy-safe; 4-bit breaks Qwen-0.5B.	Perplexity: Llama 12.04 / 12.05 / 12.73; Qwen 13.02 / 13.19 / 100.32.
4	The 7W mode is not the energy-efficient mode.	f16 energy per token is 6% to 23% higher at 7W than at 15W; decoding is 2.0 to 2.4 times slower.

Why finding 1 happens: average power is almost the same for all three formats (about 9.6 to 10.4 W at 15W), but decoding is slower with a compressed cache (for example, Llama at 16K: 22.8 tokens/s with f16 vs 18.2 with 8-bit). Unpacking the compressed values costs more GPU work than the memory reads it saves. The GPU runs at only 625 MHz (15W) or 408 MHz (7W), so compute, not memory bandwidth, is the bottleneck, and the penalty is larger at the lower clock. This answers the thesis question "do compression overheads cancel the savings?" with **yes** for this device and software.

2. Background: the thesis and why the edge

The thesis (finished May 2026) benchmarked KV-cache eviction methods (StreamingLLM and SnapKV, through NVIDIA's kvpress library) on a Tesla T4 GPU, with energy read through NVML. It produced 252 measured configurations on Qwen2.5-0.5B and Llama-3.2-1B. Quantization methods such as KIVI could not run there, because their kernels need an Ampere-class GPU (sm_80 or newer) and the T4 is Turing (sm_75).

A re-analysis on 27 September 2026 found several problems in the thesis data (Appendix D), for example that perplexity could not see kvpress compression and that joules per token was skewed by early stopping. The paper direction chosen on 29 September is *KV-cache management comparison on an edge device (Jetson Orin Nano) with energy consumption*. The Orin Nano is Ampere (sm_87), has only 8 GB of memory shared by CPU and GPU, much lower memory bandwidth than a server GPU, and selectable power modes, so it is a different regime from the T4. The starting hypothesis was that compression would save *more* energy on the edge because decoding there is limited by memory bandwidth. The results

contradict that hypothesis for this software stack.

3. What happened, step by step

Date	Step	Details
27 Sep	Catch-up and audit	Read the proposal, final report, notebooks and both result files; re-analysed all 252 thesis configurations and listed 7 problems (Appendix D). Wrote a study plan.
29 Sep	Paper direction	Chose the edge topic (Jetson Orin Nano). Wrote a 12-week plan with four research questions.
1 Oct	Plan deck	Made a 5-slide plan presentation and explained each slide.
14 Oct	Code written	Created the thesis2 folder: power sampler (power.py), benchmark (bench_kv.py), setup, environment check and perplexity scripts, plotting script and README.
14 Oct	Tested on the Mac	Homebrew could not install llama.cpp (Xcode licence not accepted), so the official llama.cpp macOS build b11627 was used. Ran the full benchmark with simulated power readings: exact prompt and output lengths confirmed, plots produced. Found that new llama.cpp no longer logs the KV cache size, so the size is now calculated from the model file header.
14 Oct	First Jetson contact	Over USB-C the Jetson showed a serial console with a login prompt for an existing account on host "ranjotsingh"; the USB network link was inactive and SSH was not available.
19 Oct	Access set up	With a monitor attached, the user account koushik was used, openssh-server was installed and enabled, the board joined Wi-Fi, and the Mac's SSH key was copied (ssh-copy-id).
19 Oct	Environment check	Found L4T R39.2 / Ubuntu 24.04, 15W mode, no CUDA compiler, cmake or tmux. The power sensor worked at once (about 18 samples/s, 3.56 W idle).
19 Oct	Packages installed	cuda-toolkit-13-2, cmake and tmux (after a typo in a package name stopped the first attempt; see Appendix C). setup.sh was changed to skip apt when tools are present.
19 Oct	Build and downloads	setup.sh compiled llama.cpp (commit 7adf160) with CUDA for sm_87 and downloaded both models, the prompt text and WikiText-2. Finished at 19:03.
19 Oct	Quick test	6 runs (Llama, 1K and 4K, 15W) at 19:04: GPU use confirmed (CUDA0: Orin, prompt processing about 1,600 tokens/s), exactly 128 tokens per run.
19 Oct	15W benchmark	72 runs from 19:06 to 19:21, then the perplexity tests until about 19:24.
19 Oct	Switch to 7W	nvpmodel -m 1 needs a reboot; done with --force. GPU limit dropped to 408 MHz and 2 CPU cores went offline.
19 Oct	7W benchmark	72 runs from 19:44 to 20:15.
19 Oct	Back to 15W	nvpmodel -m 0 --force (reboot). Results copied to the Mac, plots and tables made, RESULTS.md written.

4. Hardware

Component	Detail
Board	NVIDIA Jetson Orin Nano Developer Kit (8 GB); hostname "ranjotsingh"; benchmarks run as user koushik
CPU	6 × Arm Cortex-A78AE

Component	Detail
GPU	NVIDIA Ampere architecture, compute capability 8.7 (sm_87), 1024 CUDA cores, 32 tensor cores
Memory	8 GB LPDDR5 shared by CPU and GPU; 7,486 MB visible to Linux; no swap configured
Storage	microSD card, 113 GB root partition (/dev/mmcblk0p1), about 94 GB free before setup
Power sensor	Onboard TI INA3221 with three channels: VDD_IN, VDD_CPU_GPU_CV, VDD_SOC
Cooling	Stock dev-kit heatsink and fan
Control	SSH over Wi-Fi from a MacBook (Apple M2) at 10.0.0.x on the home network

Power mode	nvpmodel ID	GPU max clock	CPU cores online	Idle VDD_IN	Avg. VDD_IN during requests
15W (default)	0	624.75 MHz	6 (0-5)	3.57 W	about 8.2 to 10.4 W
7W	1	408 MHz	4 (0-3)	3.41 W	about 5.9 to 6.6 W

Only these two modes are defined in /etc/nvpmodel.conf on this software image; the 25W / MAXN Super mode is not listed. Switching modes requires a reboot.

5. Software, libraries and tools

On the Jetson

Item	Version / setting
Operating system	Ubuntu 24.04.4 LTS
Linux kernel	6.8.12-1021-tegra
Jetson Linux (L4T)	R39.2.0 (nvidia-l4t-core 39.2.0-20260601), JetPack 7 line
CUDA	cuda-toolkit-13-2 13.2.2-1; nvcc 13.2.86; libcublas 13.4.1.3
Compiler / build	GCC 13.3.0; CMake 3.28.3; GNU Make
Inference engine	llama.cpp, git commit 7adf160 (reports version 0.7.0-dev), cloned from github.com/ggml-org/llama.cpp
llama.cpp build flags	-DGGML_CUDA=ON -DCMAKE_CUDA_ARCHITECTURES=87 -DCMAKE_BUILD_TYPE=Release -DLLAMA_CURL=OFF; targets llama-server, llama-perplexity, llama-cli; built with -j4
Python	3.12.3; benchmark code uses the standard library only (urllib, json, threading, subprocess)
Power readout	Linux hwmon sysfs files of the ina3221 driver; tegrastats available as fallback
Session tool	tmux, so long runs survive a dropped SSH connection

On the Mac (analysis)

Item	Version / setting
Python	3.14.4 in a virtual environment
Plotting	matplotlib 3.11.2 (analysis/plot.py)
Report	reportlab 5.0.1 (this PDF)

Item	Version / setting
Dry-run engine	llama.cpp macOS arm64 release b11627, used only to test the pipeline before the Jetson
File transfer	rsync over SSH

Own code (thesis2 folder)

File	What it does
jetson/power.py	Finds INA3221 rails, thermal zones and the GPU clock file; samples power every 50 ms in a background thread; integrates energy over any time window; also a live sanity check (--watch).
jetson/bench_kv.py	Measures idle power, starts llama-server per model / round / KV format, sends fixed-length requests, records energy, speed, temperatures and clock, computes KV size from the model file, writes one JSON line per run.
jetson/setup.sh	Builds llama.cpp with CUDA and downloads models, prompt text and WikiText-2 (skips finished steps).
jetson/ppl_kv.sh	Runs llama-perplexity for each model and KV format and writes a CSV.
jetson/check_env.sh	Prints board, L4T, power mode, CUDA, memory, disk, tools and power sensors.
analysis/plot.py	Turns the JSONL and CSV into six charts and a summary table.

6. Models

Property	Llama-3.2-1B-Instruct	Qwen2.5-0.5B-Instruct
Maker	Meta	Alibaba (Qwen team)
File	Llama-3.2-1B-Instruct-Q8_0.gguf	qwen2.5-0.5b-instruct-q8_0.gguf
Downloaded from	huggingface.co/bartowski/Llama-3.2-1B-Instruct-GGUF	huggingface.co/Qwen/Qwen2.5-0.5B-Instruct-GGUF
File size	1,321,083,008 bytes (1.32 GB)	675,710,816 bytes (0.68 GB)
Weight format	Q8_0 (8-bit)	Q8_0 (8-bit)
Layers	16	24
Attention (query) heads	32	14
KV heads (grouped-query attention)	8	2
Head size	64	64
Hidden size	2,048	896
KV cache per token, f16	32 KiB	12 KiB

Architecture values were read from each GGUF file header. KV bytes per token = layers × KV heads × head size × 2 (K and V) × bytes per value.

7. KV cache formats compared

Format	How values are stored	Bytes per value	Size vs f16
f16 (baseline)	16-bit floating point, no compression	2	100%
q8_0 (8-bit)	Blocks of 32 values: 32 × 8-bit integers plus one fp16 scale (34 bytes per block)	1.0625	53%
q4_0 (4-bit)	Blocks of 32 values: 32 × 4-bit integers plus one fp16 scale (18 bytes per block)	0.5625	28%

K and V always used the same format, set with the llama.cpp flags -ctk and -ctv. A quantized V cache needs flash attention, so flash attention was turned on (-fa on) in **every** run, including f16, to keep the comparison fair. This is the same family of technique as KIVI (KV-cache quantization), which the T4 could not run.

8. How energy was measured

• **Sensor:** the onboard INA3221 reports bus voltage (mV) and current (mA) for three rails. Power per rail = mV × mA / 1,000,000 W.

• **Rails:** VDD_IN (input to the whole module: CPU, GPU, memory and the rest) is used for every headline number. VDD_CPU_GPU_CV and VDD_SOC were also recorded for every run.

• **Sampling:** a Python background thread reads the sysfs files every 50 ms; with read time this gives about 18 samples per second. Each sample also records available memory.

• **Idle baseline:** before each benchmark set, 10 seconds with nothing running; the mean watts per rail is the idle power (15W set: VDD_IN 3.572 W; 7W set: 3.410 W).

• **Integration:** trapezoid rule over the samples between sending a request and receiving the reply, with the first and last sample extended flat to the window edges.

• **Net energy:** $E_{net} = \int P\,dt - P_{idle} \times$ (request duration).

• **Prefill / decode split:** prefill is taken to end at send time + llama.cpp's own prompt-processing time (timings.prompt_ms); the rest of the request is decode. This split is approximate.

• **Headline metric:** decode energy per token = net decode energy ÷ 128 generated tokens.

• **Also recorded per run:** prefill and decode speed (from llama.cpp timings), GPU temperature before and after, GPU clock, minimum available memory, number of power samples.

9. Benchmark protocol and parameters

Setting	Value
Server program	llama-server from the same build
Server flags	-m -c 16576 -ngl 99 -np 1 -ctk -ctv -fa on --host 127.0.0.1 --port
Meaning of flags	context window 16,576 tokens (largest prompt 16,384 + 128 generated + 64 spare); all layers on the GPU; one request slot; KV format for K and V; flash attention on
Prompt text	Pride and Prejudice by Jane Austen, Project Gutenberg eBook #1342 (772,386 bytes)

Setting	Value
Prompt construction	The text is tokenized once per model through the server's /tokenize endpoint; the first N token IDs are sent directly, so prompt lengths are exact
Prompt lengths N	1,024; 4,096; 8,192; 16,384 tokens
Request	POST /completion with n_predict = 128, ignore_eos = true, temperature = 0 (greedy), cache_prompt = false, stream = false
Repeats	3 rounds per power mode
Order	Within each round, the order of KV formats is shuffled, and within each server the order of prompt lengths is shuffled
Server restarts	A fresh server for every model, round and KV format: 2 × 3 × 3 = 18 per power mode
Warm-up	One 16-token request with the 1K prompt after every server start, not measured
Cool-down	2 seconds before each measured request
Clocks	Not locked with jetson_clocks; the GPU clock was nevertheless constant at the mode maximum in all runs
Total	2 models × 3 formats × 4 lengths × 3 rounds = 72 runs per power mode; 144 overall
Wall time	15W set: 19:06 to 19:21 (15 min). 7W set: 19:44 to 20:15 (31 min)

These choices directly fix three problems found in the thesis data: a fixed output length removes the early-stop skew in joules per token; shuffled order removes the "baseline always first" bias; and turning off prompt caching makes sure the prompt is really processed in every run.

10. Accuracy test

Setting	Value
Tool	llama-perplexity from the same build
Command	llama-perplexity -m -f data/wiki.test.raw -c 2048 --chunks 20 -ngl 99 -ctk -ctv -fa on
Dataset	WikiText-2 raw, test split (wikitext-2-raw-v1 from the ggml-org/ci mirror on Hugging Face), 1,290,590 bytes
Amount scored	20 chunks × 2,048 tokens = 40,960 tokens
Runs	2 models × 3 formats = 6, at 15W (accuracy does not depend on the power mode)
Metric	Perplexity, lower is better, with llama.cpp's ± error estimate

In llama.cpp, attention reads K and V back from the stored (compressed) cache, so this perplexity **does** reflect the compression. In the thesis, kvpress compressed the cache only after the forward pass, so its perplexity could not.

11. Results

11.1 Energy: quantized KV caches cost more per token

Change in decode energy per token compared with f16 (positive = uses **more** energy). Mean of 3 rounds, VDD_IN, idle subtracted.

Mode	Context	Llama 8-bit	Llama 4-bit	Qwen 8-bit	Qwen 4-bit
15W	1K	+0.5%	−0.6%	+5.3%	+3.2%
15W	4K	+3.5%	+2.3%	+14.8%	+12.7%
15W	8K	+17.3%	+12.0%	+32.9%	+25.0%
15W	16K	+35.4%	+28.5%	+58.9%	+50.5%
7W	1K	+6.2%	+2.3%	+8.2%	+2.4%
7W	4K	+13.8%	+11.5%	+16.1%	+14.8%
7W	8K	+30.2%	+25.1%	+44.7%	+25.9%
7W	16K	+61.1%	+51.5%	+108.9%	+73.6%

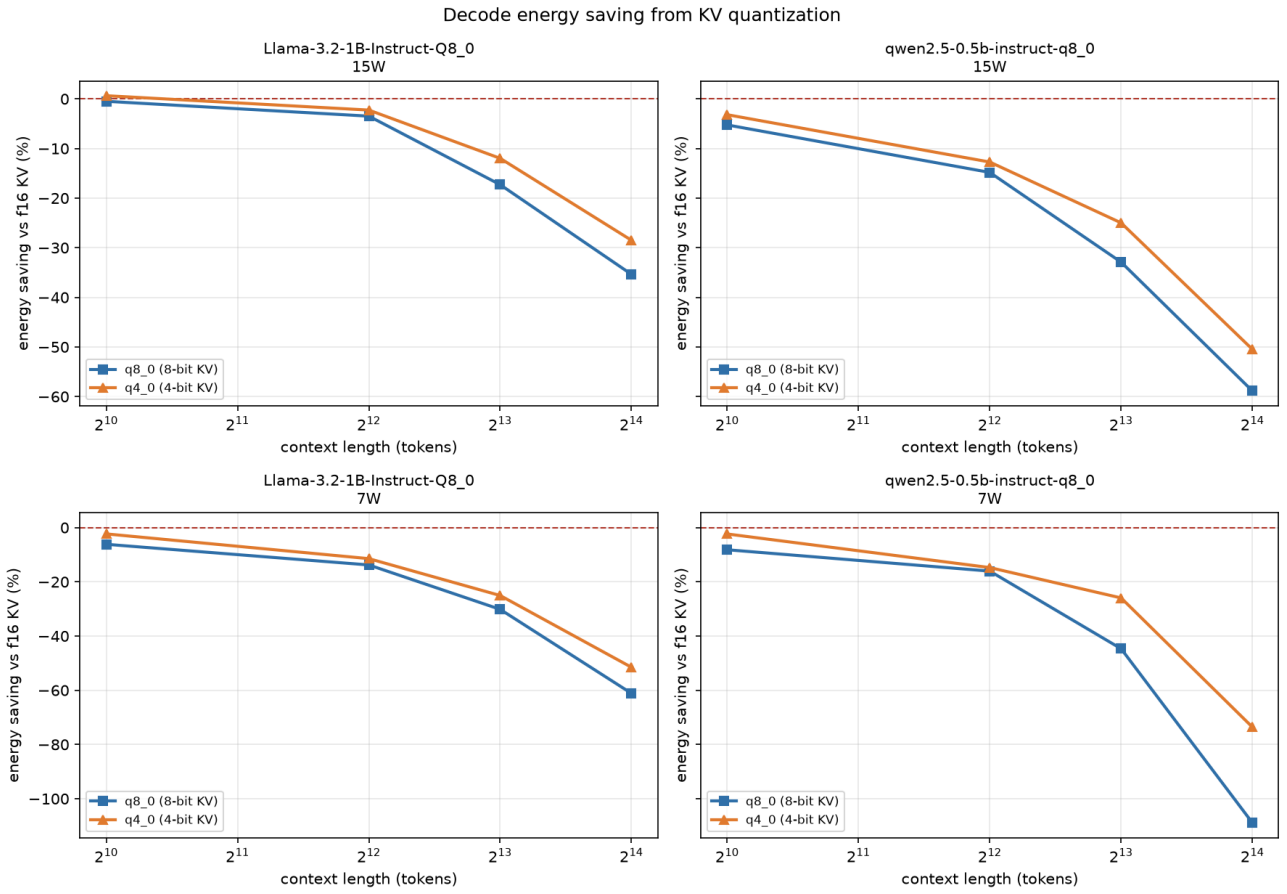


Figure 1. Decode energy saving versus f16 (negative = more energy). Top row 15W, bottom row 7W. File: plots/02_saving_vs_f16.png

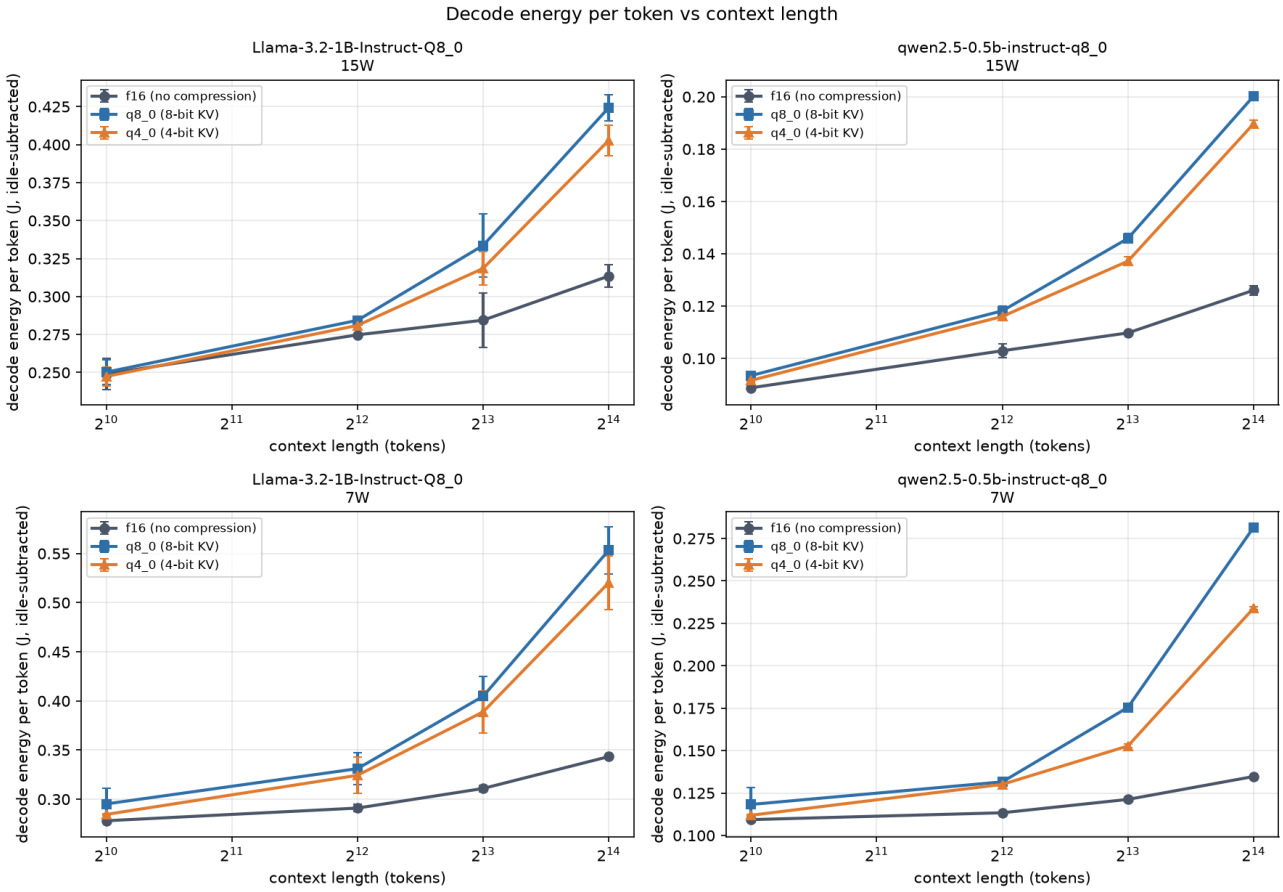


Figure 2. Decode energy per token (J) against context length, with ±1 standard deviation across the 3 rounds. File: plots/01_decode_j_per_token.png

11.2 Why: same power, slower decoding

At 16K tokens	f16	8-bit	4-bit
Llama decode speed, 15W (tokens/s)	22.8	18.2	17.7
Llama decode speed, 7W	10.3	7.2	6.7
Qwen decode speed, 15W	50.8	34.0	32.5
Qwen decode speed, 7W	24.8	13.6	12.8
Llama average power, 15W (W)	10.07	10.37	10.11
Llama prefill speed, 15W (tokens/s)	1,291	1,249	1,232

Energy is power multiplied by time. Power changes by at most about 3% between formats, so the extra energy comes almost entirely from the extra time. Unpacking 8-bit or 4-bit values inside the attention kernel adds GPU work; at 625 MHz or 408 MHz that work costs more than the memory traffic it saves, and the penalty grows at the lower clock. This explanation fits the data but is not yet proven; it may also reflect how llama.cpp's quantized flash-attention kernels are written for this GPU (see Limitations).

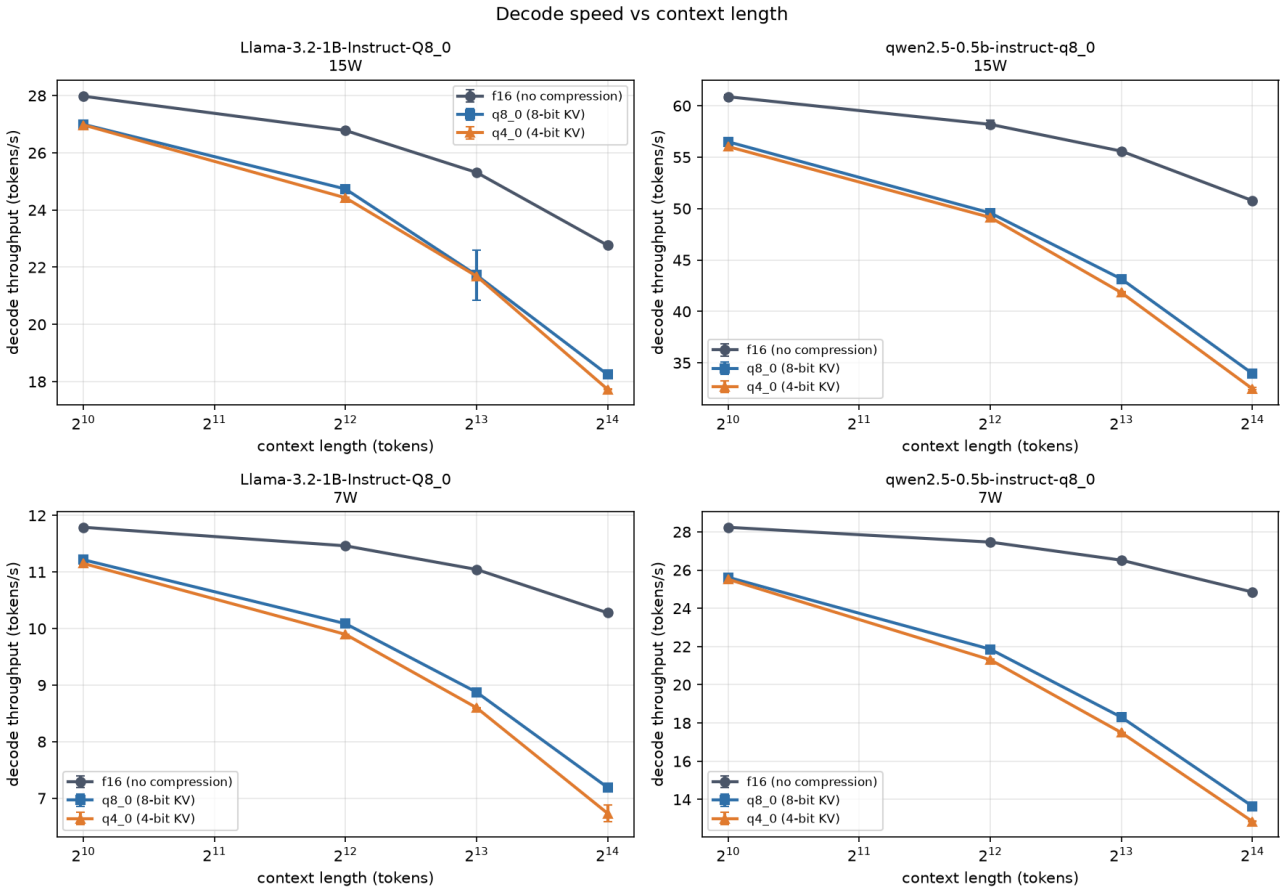


Figure 3. Decode speed (tokens/s) against context length. File: plots/03_decode_tps.png

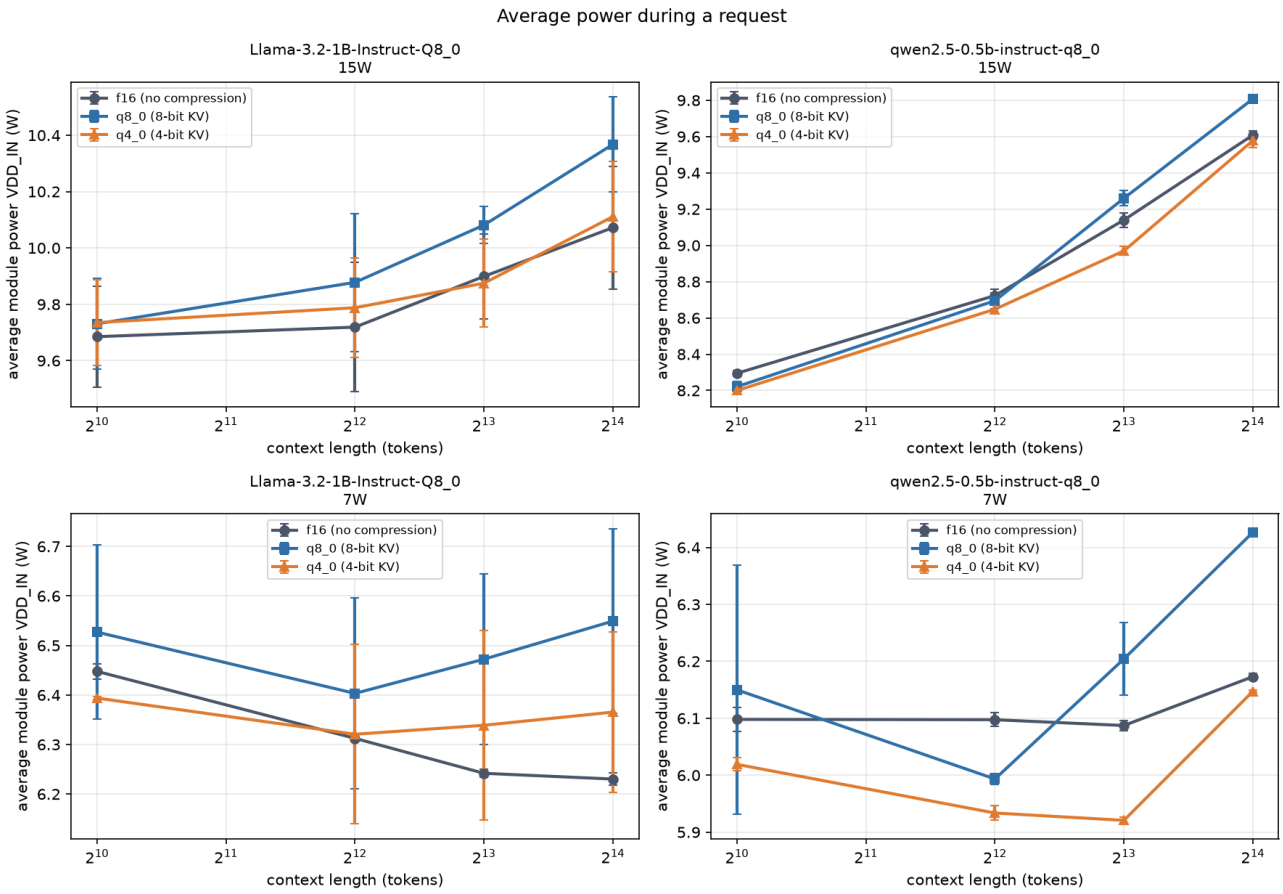


Figure 4. Average module power (VDD_IN) during a request. File: plots/05_avg_power.png

11.3 Where the energy of a request goes

One request = a 16,384-token prompt plus 128 generated tokens. Net energy in joules, mean of 3 rounds.

Model	KV	15W prefill	15W decode	15W prefill share	7W prefill	7W decode	7W prefill share
Llama-1B	f16	79.0	40.1	66%	91.7	44.0	68%
Llama-1B	8-bit	82.3	54.3	60%	99.4	70.8	58%
Llama-1B	4-bit	82.4	51.6	62%	99.6	66.6	60%
Qwen-0.5B	f16	44.7	16.1	73%	52.8	17.2	75%
Qwen-0.5B	8-bit	46.4	25.6	64%	54.5	36.0	60%
Qwen-0.5B	4-bit	46.6	24.3	66%	54.6	29.9	65%

With long prompts and short answers, reading the prompt (prefill) takes 58% to 75% of the request energy. Compression adds 3% to 9% to prefill energy but 29% to 109% to decode energy.

11.4 Memory: the saving is real

KV cache at 16,512 tokens (16,384 + 128)	f16	8-bit	4-bit
Llama-3.2-1B	516 MiB	274 MiB	145 MiB
Qwen2.5-0.5B	194 MiB	103 MiB	54 MiB

Calculated exactly from the model architecture. The KV cache grows linearly with context: at Llama's full 128K context, f16 would need about 4 GiB of the board's 8 GB and 4-bit about 1.1 GiB (same formula, not measured). On an 8 GB board shared with the operating system (3.6 to 6.5 GB was free before the runs), this is what lets longer contexts or larger models fit.

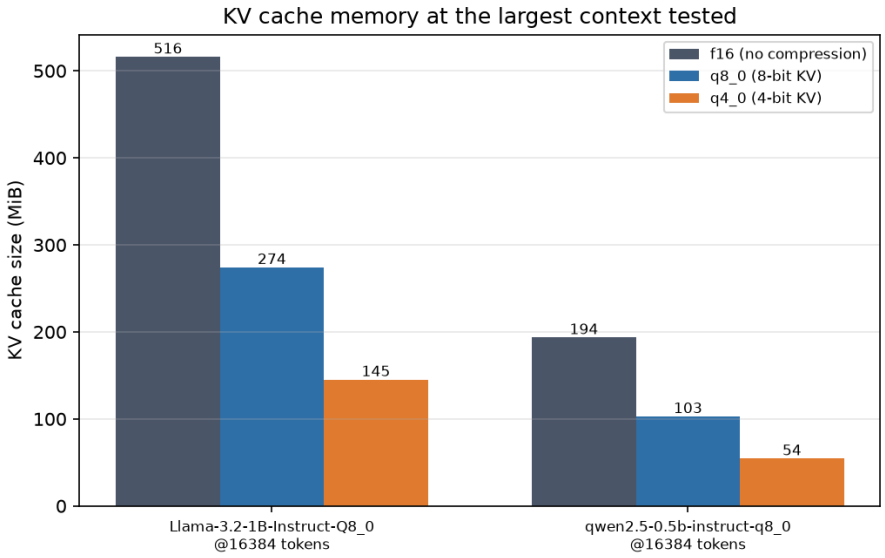


Figure 5. KV cache size at the largest context tested. File: plots/06_kv_cache_mib.png

11.5 Accuracy: 8-bit is safe, 4-bit breaks Qwen

Model	KV format	WikiText-2 perplexity	Change vs f16
Llama-1B	f16	12.0443 ± 0.2285	baseline
Llama-1B	8-bit	12.0489 ± 0.2286	+0.04% (no change)
Llama-1B	4-bit	12.7315 ± 0.2417	+5.7% (small loss)
Qwen-0.5B	f16	13.0188 ± 0.2533	baseline
Qwen-0.5B	8-bit	13.1853 ± 0.2572	+1.3% (small loss)
Qwen-0.5B	4-bit	100.3182 ± 2.5995	+671% (model breaks)

A likely reason for the Qwen failure is that it has only 2 KV heads, each shared by 7 query heads, so every rounding error in the cache affects many attention heads at once. This was not tested directly.

11.6 Power modes: 7W draws less power but costs more energy

f16 decode J/token	1K	4K	8K	16K
Llama 15W → 7W	0.249 → 0.278	0.275 → 0.291	0.284 → 0.311	0.314 → 0.343
Change	+11.7%	+5.9%	+9.3%	+9.5%

f16 decode J/token	1K	4K	8K	16K
Qwen 15W → 7W	0.089 → 0.109	0.103 → 0.113	0.110 → 0.121	0.126 → 0.135
Change	+23.1%	+10.2%	+10.6%	+6.9%

At 7W the module draws about 6 W instead of 8 to 10 W, but decoding is 2.0 to 2.4 times slower, so each token costs more energy. Finishing quickly and returning to idle beats running slowly ("race to idle"). 7W is only worth using under a hard power or temperature limit.

11.7 Full table

Every number (decode J/token ± standard deviation, saving vs f16, decode and prefill speed, net request energy, average power, perplexity, number of rounds) for all 48 settings is in plots/summary.md. Raw per-run data: results/kv_20261019_190621.jsonl (15W), results/kv_20261019_194427.jsonl (7W), results/ppl_20261019_192054.csv.

12. Data quality

Check	15W set	7W set
Runs completed	72 of 72	72 of 72
Tokens generated per run	exactly 128 in every run	exactly 128 in every run
Prompt lengths	exactly 1,024 / 4,096 / 8,192 / 16,384	same
Power samples per run	45 to 375	100 to 1,038
Spread across 3 rounds (std ÷ mean of decode J/token)	max 6.3%, median 1.5%; 19 of 24 settings under 3%	max 8.5%, median 1.0%; 16 of 24 settings under 3%
GPU clock at end of every run	624.75 MHz	408 MHz
Hottest GPU reading	51.7 °C (no throttling)	47.3 °C (no throttling)
Request duration	2.5 to 20.5 s	5.4 to 56.7 s

The 6-run quick test (results/kv_20261019_190420.jsonl) checked the pipeline and is not used in the analysis.

13. Limitations

- **One software stack.** The energy penalty may come from how llama.cpp's quantized flash-attention kernels run on this GPU, rather than from KV quantization in general. A second inference engine is needed to separate the two.
- **Whole-module power.** VDD_IN includes CPU, memory and other parts of the module, so it is not directly comparable with the GPU-board-only NVML numbers from the T4 thesis.
- **Small models.** At 0.5B and 1B parameters the KV cache is small next to the weights (Llama at 16K: 516 MiB of cache vs 1.32 GB of weights).
- **One method family.** Only quantization was tested; eviction methods (StreamingLLM, SnapKV, H2O) were not run on the Jetson yet.

- **Clocks not locked.** jetson_clocks was not used; the GPU clock was constant, but memory and CPU clocks were not controlled.
- **Approximate split.** Prefill and decode energy are separated using llama.cpp's timing rather than a hardware marker.
- **Accuracy proxy.** Perplexity only; no task scores (question answering, summarization).
- **Only two power modes.** The faster 25W / Super mode was not available on this image.

14. Next steps

Step	Why
Eviction methods (StreamingLLM, SnapKV) on the Jetson	Compare the other compression family on the same hardware
A second inference engine	Check whether the energy penalty is specific to llama.cpp
Llama-3.2-3B (Q4_K_M) and longer contexts (32K+)	Make the KV cache a bigger share of memory traffic; reach the 8 GB limit
Lock clocks; enable 25W / Super mode if the firmware allows	Remove clock variation and test a faster GPU clock
Task accuracy (LongBench, QA exact match, ROUGE)	Measure real task quality, not only perplexity
Same harness on the T4 and an RTX 5090	Edge vs server comparison for the paper (research question 4)

If the energy penalty shrinks at a higher GPU clock or with a larger model, that supports the compute-bound explanation. If it stays the same with a second engine, the result is about KV quantization itself rather than one implementation.

Appendix A. Reproduction commands

One-time installs on the Jetson (need the sudo password):

```
sudo apt update
sudo apt install -y openssh-server && sudo systemctl enable --now ssh
sudo apt install -y cuda-toolkit-13-2 cmake tmux
```

From the Mac, give SSH access and copy the code:

```
ssh-copy-id <user>@<jetson-ip>
rsync -a --exclude results ~/project/school/Thesis/thesis2 <user>@<jetson-ip>:~/
```

On the Jetson, in ~/thesis2:

```
bash jetson/check_env.sh                     # board, L4T, power mode, sensors
bash jetson/setup.sh                         # build llama.cpp, download models + data
python3 jetson/power.py --watch 10           # power sensor sanity check
python3 jetson/bench_kv.py --quick --tag test    # 6-run pipeline check (~2 min)
python3 jetson/bench_kv.py --tag 15W         # 72 runs (~15 min)
bash jetson/ppl_kv.sh                        # 6 perplexity tests (~3 min)
sudo nvpmodel -m 1 --force                   # switch to 7W (reboots)
python3 jetson/bench_kv.py --tag 7W          # 72 runs (~31 min)
sudo nvpmodel -m 0 --force                   # back to 15W (reboots)
```

Back on the Mac, in thesis2/:

```
rsync -a <user>@<jetson-ip>:~/thesis2/results/ results/
pip3 install matplotlib
python3 analysis/plot.py results/kv_20261019_190621.jsonl results/kv_20261019_194427.jsonl \
        --ppl results/ppl_20261019_192054.csv --out plots
```

Useful benchmark flags: --contexts 1024,4096,8192,16384,32768 · --gen 256 · --rounds 5 · --models models/x.gguf · --rail VDD_CPU_GPU_CV · --idle-secs 10 · --cooldown 2.

Appendix B. Files produced

Path (inside Thesis/thesis2)	Content
README.md	Step-by-step guide from flashing the Jetson to plotting
RESULTS.md	One-page findings with tables and caveats
Jetson_KV_Report.pdf	This report
jetson/*.py, jetson/*.sh	Benchmark, power sampler, setup, environment check, perplexity
analysis/plot.py	Plotting and summary table
results/kv_20261019_190621.jsonl (+ .meta.json)	15W set: 72 runs, one JSON object per run; metadata incl. idle power
results/kv_20261019_194427.jsonl (+ .meta.json)	7W set: 72 runs
results/kv_20261019_190420.jsonl	6-run quick test (not analysed)
results/ppl_20261019_192054.csv	Perplexity for 2 models × 3 formats
results/logs/	One llama-server log per server start and one llama-perplexity log per test

Path (inside Thesis/thesis2)	Content
plots/01 … 06 *.png	Decode J/token, saving vs f16, decode speed, prefill speed, average power, KV cache size
plots/summary.md	Every number for all 48 settings

Each JSON line holds: model, kv_type, context_len, round, tag, nvpmodel, kv_alloc_mib, kv_used_mib, server_mem_mb, idle_w, rail, power_backend, prompt_tokens, gen_tokens, latency_s, prefill_s, decode_s, prefill_tps, decode_tps, gpu_temp_start, gpu_temp_end, gpu_freq_mhz_end, min_mem_available_mb, energy and average power for each rail, n_samples, energy_j, energy_net_j, prefill_energy_net_j, decode_energy_net_j, decode_j_per_token, total_j_per_token.

Appendix C. Problems hit along the way and how they were fixed

Problem	Cause	Fix
Jetson not reachable from the Mac	USB network link inactive; SSH server not installed	Installed and enabled openssh-server; used Wi-Fi
No login for the existing account	Board had been set up under another account	Used the koushik account with a monitor attached
"Unable to locate libcur14-openssl-dev"	Typo: digit 1 instead of the letter l; apt then installed nothing	Dropped the package (not needed; LLAMA_CURL=OFF)
Same error again	The old command was re-run from shell history (up arrow)	Typed the shorter command fresh
setup.sh needed sudo	It always ran apt first	Now skips apt when git, cmake, g++, curl and unzip are present
"bad parameter for -m"	Most likely a mistyped mode number	Ran sudo nvpmodel -m 1 with the correct syntax
nvpmodel asked to reboot and aborted	Switching to 7W requires a reboot; no answer given	Added --force so it reboots automatically
KV cache size missing from logs	Newer llama.cpp no longer prints it	Calculated from the GGUF header (exact)
Homebrew install failed on the Mac	Xcode licence not accepted	Used the official llama.cpp macOS release for the dry run
matplotlib missing on the Mac	Not installed system-wide	Used a Python virtual environment
Charts hard to read with 4 panels in a row	One row per chart	Changed to a grid: one row per power mode

Security note: the Jetson sudo password was shared in the chat so that the power mode could be switched. It was used only for nvpmodel and was not saved to any file. Change it with passwd on the Jetson.

Appendix D. The thesis-data audit (27 September)

#	Problem in the thesis data	How the Jetson harness avoids it
1	Perplexity could not see kvpress compression (it compresses after the forward pass)	llama.cpp attention reads the compressed cache, so perplexity reflects it
2	Joules per token skewed by early end-of-sequence in 25 of 252 configurations	ignore_eos: exactly 128 tokens every run
3	The "break-even near batch size 4" claim held for Llama only	Not applicable (single request at a time); claims checked per model
4	Baseline always ran first (order and thermal bias)	KV format and context order shuffled every round
5	T4 sat at its ~70 W power cap from batch size 4	GPU clock and temperature logged; no throttling seen
6	15–27% savings larger than memory traffic could explain	Mechanism checked with speed and power separately
7	Fewer runs than planned; KIVI, H2O, larger models and long contexts missing	Quantization now tested; contexts up to 16K

Appendix E. Glossary

Term	Meaning
KV cache	Stored keys and values of earlier tokens, reused so attention does not recompute them for every new token
Prefill	Processing the whole prompt at once, before the first output token
Decode	Generating output tokens one at a time; each step reads the whole KV cache
f16 / q8_0 / q4_0	16-bit float / 8-bit block-quantized / 4-bit block-quantized storage
Flash attention	An attention algorithm that avoids building the full attention matrix; needed for a quantized V cache in llama.cpp
Perplexity	Exponential of the average negative log-likelihood per token; lower means better prediction
VDD_IN	The INA3221 channel measuring power into the whole Jetson module
nvpmodel	NVIDIA tool that switches Jetson power modes (clock and core limits)
jetson_clocks	NVIDIA tool that pins clocks at the maximum for the current mode
L4T / JetPack	Jetson Linux (board support package) / NVIDIA's full Jetson software bundle
GGUF	The model file format used by llama.cpp
Race to idle	Finishing work fast at higher power and then idling often uses less energy than running slowly